

Implementing Discrete Fourier Transformation and the Fast Fourier Transformation

NOTES

- FFT is a quicker method of the discrete Fourier Transformation, which is used in Audio to convert from the Time-Domain to Frequency Domain Analysis.
- After applying the FFT you get an array of complex numbers , the magnitude of the frequency bin and the phase offset of that frequency bin as well
- The width of frequency bins and therefore the resolution of the Frequency Domain Analysis is equal to the sample rate divided by the FFT size , the FFT size is the number of samples used.
- By increasing the FFT Size you also increase the resolution, additionally you also increase the computing power needed to complete.
- For real time audio applications the computational time needs to be at a reasonable level to which it does not hinder performance and use of the device.
- To allow for a detailed view of the data , we can hold a buffer of previous values for example 2048 samples and update by replacing the oldest values each time with the new 256 Samples , this increases the resolution of the device.
Overlapping the data this way means you don't have to wait for a new buffer to refill which would cause a noticeable delay.
- Windowing functions are also applied to the sampled data, as the data is continuous if from one sampled buffer to the next there was a jump from one value to the next this would cause unwanted error, of false spectral leakage. A windowing function is an array of values equal in size to the FFT size which fades each side of the samples to 0 , this is a trade-off as doing this affects the results of the data as well
- $$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-i \cdot 2\pi \cdot k \cdot n / N}$$
- Above is the equation for the FFT at frequency bin k , $x[n]$ is the input sample , $e^{-i \cdot 2\pi \cdot k \cdot n / N}$, represents a complex sine wave at frequency k. The unit circle encodes both sine and cosine waves as you move along the unit circle the

horizontal position traces out a cosine wave and its vertical position traces out a sine wave.

- $e^{-i \cdot 2\pi \cdot k \cdot n / N}$ uses Euler's equation of $e^{i\theta} = \cos(\theta) + i \cdot \sin(\theta)$, a complex number is a combination of a normal number and a multiple of i , $i = \sqrt{-1}$. If you have a point on the unit circle at an angle θ its position on the circle is $x = \cos(\theta)$, $y = \sin(\theta)$. Multiplying by i rotates the position of a number on xy plane by 90 degrees. By multiplying the i value in the exponent we scale this movement from 90 degrees to the amount we want to step by to place the next point.

- For a signal you trace around the circle moving at an angle step of $\frac{2\pi}{fftSize} \cdot k$,

where k represents the current bin, for each bin you trace around the circle placing the next sample on the circle with a weight equal to its signed level, the current bin that you are on controls how many rotations of the circle is completed, the distance by calculating the centre of mass for all of the points depending on how far from the centre gives you the magnitude for that specific frequency to calculate the phase relation of the sine at that specific frequency you measure the angle away from 0 degrees.

- What happens at certain frequency bins? At frequency bins with high magnitude the points clump to one side causing the centre of mass to move away from the centre, at frequency bins where there is little to no magnitude the points spread evenly and cancel out which means that the centre of mass is close to the centre.
- FFT implementation vs DFT implementation, to implement a DFT you have to loop through all of the frequency bins and perform the calculation on each of them, you can then use the calculated complex number to find the magnitude and phase of that frequency, this method is computationally expensive as it requires thousands of trigonometric calculations each run. The FFT aims to solve this by being less expensive, it works by splitting the buffer into odd and even indices, it does this because the trigonometric values for them are related to each other. The FFT splits the sample group into odd and even integer positions and then splits those groups again, repeating this until you have pairs as splitting into single factors returns the factor itself, returning as is with no computation, this is the stopping condition for the recursion. The relation of samples comes from here, after splitting the group you have the samples in pairs of N and $N + (Nsize / 2)$, these samples are related in that the sine and cosine coefficient are equal and the difference in the equation is its sign. The results of computing the DFT from here are combined together, at the bottom level of pairs you get the weighting of two

sample positions , after combining you get four , you continue this combination until getting back to the original buffer size , and calculating the full frequency spectrum. At the bottom level of pairs the related and computed using the

butterfly operation ,
$$X[k] = E[k] + W \cdot O[k]$$

$$X[k + N/2] = E[k] - W \cdot O[k]$$
 For one Twiddle factor [W]

you can calculate two values just by changing the sign used, the twiddle factor for each level of the FFT is calculated using , $W_k = e^{-i \cdot 2\pi \cdot \frac{k}{N}}$. To then combine the pairs together with another pair, you use a twiddle factor where N , which is the levels local size. At each level of combination you double the frequency resolution. Taking for example the second stage of combination of an 8 step buffer , combining [0,4] and [2,6] , the final results of the combination come from

$$\mathbf{k=0:} \quad X[0] = E[0] + W_0 \cdot O[0]$$

$$X[2] = E[0] - W_0 \cdot O[0]$$

the equations , $\mathbf{k=1:} \quad X[1] = E[1] + W_1 \cdot O[1]$, from this you can see how in

$$X[3] = E[1] - W_1 \cdot O[1]$$

the 4 step buffer the results of the pairs are related and combined, E[0] would be the result of 0 and E[1] is the result of 4 , O[0] is the result of 2 and O[1] is the result of 6. At each level E and O are not the raw samples but the complex number outputs from the butterfly operations below. The butterfly equation is identical are every stage, only the twiddle factor changes as the local level size increases.

- The split groups share values because $W_{k+N/2} = -W_k$ which is derived from the unit circle.
- To implement the FFT, a circular buffer and a structure to hold complex numbers is required.

CODE IMPLEMENTATION

The FFT class has 3 functions prepare , process and calcFFT. The prepare function sets the FFT size , sample rate and windowing function , the process block takes inputted audio passes it through to a circular buffer before ordering and windowing the buffer so that the samples are processed in the correct order. The calcFFT is then run passing its results to an array that holds the complex numbers.

CalcFFT function: initially a base case which checks if the input.size is equal to 1 , this is the bottom of the recursion in this case the function returns the value itself as a complex number which begins the process of the function reiterating back up and calculating the next value. If this base case is not met the vector of

samples are split into even and odd positions and then are processed again by the calcFFT and stored as two vectors of complex numbers E and O (even and odd). The process remains here until the Even and Odd both fully reach the base case and begin the process of butterfly equations. A holder variable, stores a vector of Complex numbers of inputs.size() to hold the results of the processes. To process the values a for loop is initiated, the angle for that specific bin index using $-\frac{2\pi k}{N}$, the angle is then used to calculate the twiddle factor, W. Where the imaginary value is equal to $\sin(\theta)$ and the real value is equal to $\cos(\theta)$. A temporary complex value is made to hold the results of $W \cdot O[k]$, using it in,

$\text{temp}_{\text{real}} = W_{\text{real}} \cdot O[k]_{\text{real}} - W_{\text{imag}} \cdot O[k]_{\text{imag}}$
 $\text{temp}_{\text{imag}} = W_{\text{real}} \cdot O[k]_{\text{imag}} + W_{\text{imag}} \cdot O[k]_{\text{real}}$, by calculating this temporary value you reduce the computing cost in the two final results. From the single twiddle factor you get two outputs the original bin index, k, and its counter part, $k + (\frac{\text{inputSize}}{2})$, the real and imaginary values for these are calculated using the equations,

$$\text{result}[k] = \{E[k]_{\text{real}} + \text{temp}_{\text{real}}, E[k]_{\text{imag}} + \text{temp}_{\text{imag}}\}$$

$$\text{result}\left[k + \frac{N}{2}\right] = \{E[k]_{\text{real}} - \text{temp}_{\text{real}}, E[k]_{\text{imag}} - \text{temp}_{\text{imag}}\}$$

These values are then used by the next stage of recursion until the second from top level produces the final output.

K and $k + N/2$ share a twiddle factor as they are directly opposite on the unit circle, they share a twiddle factor because substituting k and $k + N/2$ into Eulers equation you get:

- $W_k = e^{-i2\pi k/N}$
- $W_{k+N/2} = e^{-i2\pi(k+N/2)/N}$
- $= e^{-i2\pi k/N} \cdot e^{-i2\pi(N/2)/N}$
- $= e^{-i2\pi k/N} \cdot e^{-i\pi}$
- $e^{-i\pi} = \cos(-\pi) + i \sin(-\pi) = -1$
- $W_{k+N/2} = e^{-i2\pi k/N} \cdot (-1) = -W_k$